

UNIVERSIDAD PRIVADA DOMINGO SAVIO

Facultad de Ingeniería

TRABAJO FINAL INTEGRADOR · INFORME TÉCNICO

Sistema web de la Asociación «Manos del Oriente»

Liquidación de socias con control de calidad y pedidos con reserva de stock, publicado con PHP y MySQL

Integrantes:

1. Daniela Cayo Mamani
2. Juan Jose Vicente Cossio
3. Ronald Salvatierra Flores
4. Yordy Tolavi Ruiz
5. Alejandro Cadario Lopez

Carrera: Ingeniería de Sistemas

Asignatura: Tecnología Web II (SIS-0151)

Docente: M.Sc. Ing. Jimmy Nataniel Requena Llorentty

Santa Cruz de la Sierra, Bolivia

26 de octubre de 2026

Índice

Resumen 4
Introducción 5
La organización y el problema 6
Objetivos medibles 7
Marco teórico 8
Integridad de datos y transacciones 8
Seguridad de aplicaciones web 8
Servicios y API 8
Metodología 9
Requerimientos 10
Diseño 11
Arquitectura 11
Modelo de datos 11
Implementación de los módulos 13
Módulo 1: liquidación y pagos con control de calidad 13
Módulo 2: pedidos de compradores con reserva de stock 14
Seguridad 17
Publicación 18
Pruebas 19
Resultados y discusión 21
Conclusiones 22
Referencias 23
Anexos 25
Anexo A. Registro de uso de inteligencia artificial 25
Anexo B. Repositorio y documentos 25

Índice de figuras y tablas

Tabla 1 *Problemas que el sistema debe resolver* 6
Tabla 2 *Objetivos e indicadores* 7
Tabla 3 *Fases del trabajo* 9
Tabla 4 *Requerimientos de los módulos nuevos y no funcionales* 10
Tabla 5 *Decisiones de diseño de la base de datos* 11
Tabla 6 *Liquidaciones calculadas a mano con los datos de demostración (Bs)* 14
Tabla 7 *Amenazas y defensas* 17
Tabla 8 *Resultados de las pruebas* 19
Tabla 9 *Objetivos frente a resultados* 21
Tabla 10 *Ubicación de los entregables* 25

Figura 1 *Modelo entidad-relación (13 tablas, 18 claves foráneas)* 11
Figura 2 *Liquidaciones vistas por la directiva, con el detalle de Rosa* 13
Figura 3 *La misma información vista por la socia: solo la suya* 14
Figura 4 *Pedidos con su estado y el stock reservado* 15
Figura 5 *Importación de un pedido JSON o XML* 16
Figura 6 *Un operador que intenta entrar a Usuarios recibe 403* 17
Figura 7 *Salida de php verificar.php* 20

Resumen

La Asociación de Confeccionistas Textiles «Manos del Oriente» (caso verosímil declarado, Chiquitanía, Bolivia) tenía dos discusiones recurrentes: socias que reclamaban que se les pagaban menos prendas de las que entregaban, y pedidos de boutiques que no se podían completar porque las prendas se habían vendido en la feria. Sobre el sistema construido durante el semestre (catálogo, MySQL, roles, ventas, reportes, API REST y AJAX) se agregaron dos módulos. El primero registra cada entrega con prendas entregadas, aprobadas y el motivo del rechazo, y calcula el saldo de cada socia como lo liquidado por lo aprobado menos lo pagado; cada socia ve solo su liquidación. El segundo registra pedidos que reservan stock, que al entregarse se convierten en venta en una sola transacción, con la restricción CHECK (reservado <= stock) como última defensa. La seguridad incluye consultas preparadas reales, contraseñas con password_hash, CSRF, sesión HttpOnly y SameSite, roles por página, CSP y archivos internos inaccesibles. Las 136 comprobaciones automáticas pasan en MariaDB y MySQL, 12 pruebas en navegador también y la auditoría del sistema en marcha da 18 de 18 en local y en una copia del hosting. El sistema queda empaquetado para InfinityFree con un script que genera la base para phpMyAdmin y contraseñas nuevas.

Palabras clave: sistema web, asociación de productores, PHP, MySQL, transacciones, seguridad, API REST.

Introducción

El Trabajo Final Integrador pide llevar el sistema del semestre a una organización real o a un caso verosímil, levantar sus requerimientos, diseñar la base de datos completa, agregar dos módulos nuevos que resuelvan una discusión real, mantener la seguridad, publicar el sistema y documentarlo (Requena Llorentty, 2026b). El eje es el emprendimiento con responsabilidad social, en línea con el Objetivo de Desarrollo Sostenible 8 (Naciones Unidas, s. f.).

Al no conseguir a tiempo una entrevista con una asociación real, el equipo construyó un caso verosímil y lo declaró así en todos los documentos. Los nombres, CI y teléfonos de las socias son ficticios. El mensaje para pedir la entrevista está listo para validar el sistema con una asociación real (Anexo B).

La organización y el problema

La asociación reúne a seis socias y socios (cinco activos) de Concepción, San Javier, San Ignacio de Velasco, San Rafael y Santa Ana de Velasco que confeccionan poleras, vestidos, abrigos y accesorios con telas y bordados de la región. La secretaría de ventas vende en ferias y a boutiques, hoteles y tiendas de Santa Cruz.

Tabla 1

Problemas que el sistema debe resolver

Situación (entrevista simulada)	Consecuencia
«Revisamos las prendas, pero no queda anotado cuántas rechazamos ni por qué»	Reclamos de pago: «me pagaron menos de lo que traje»
«La boutique encarga 10 poleras, vendemos 4 en la feria y al entregar no alcanzan»	Pedidos incompletos y compradores molestos
«El informe para la asamblea lo armamos a mano en 4 o 5 días»	Información tardía para decidir
«Las socias preguntan por WhatsApp cuánto se les debe»	La directiva responde una y otra vez lo mismo

Objetivos medibles

Tabla 2

Objetivos e indicadores

#	Objetivo	Indicador	Situación inicial	Meta
1	Liquidar sin discusiones	Reclamos de pago por trimestre	6	0
2	No comprometer prendas reservadas	Pedidos incompletos por mes	3	0
3	Informe trimestral automático	Tiempo para armarlo	4–5 días	10 minutos
4	Cada socia ve su saldo	Consultas a la directiva por mes	~20	Acceso propio

Nota. Situación inicial estimada para el caso verosímil.

Marco teórico

Integridad de datos y transacciones

Una transacción agrupa varias operaciones que se confirman todas o ninguna; InnoDB la ofrece junto con bloqueos de fila (`SELECT … FOR UPDATE`) que evitan que dos usuarios modifiquen la misma fila a la vez (Oracle, s. f.). Las restricciones CHECK y las claves foráneas hacen que la base rechace datos incoherentes aunque la aplicación falle.

Seguridad de aplicaciones web

El proyecto sigue las recomendaciones de OWASP: consultas parametrizadas contra la inyección SQL, tokens contra la falsificación de peticiones (CSRF), contraseñas con funciones de hash lentas y sesión con cookies protegidas (OWASP Foundation, s. f.-a, s. f.-b). La política de seguridad de contenido (CSP) limita de dónde se cargan los scripts (MDN Web Docs, s. f.).

Servicios y API

La API sigue el estilo REST (Fielding, 2000) y describe sus errores con *problem details* (Nottingham et al., 2023). Su contrato está en OpenAPI 3.1 (OpenAPI Initiative, 2021).

Metodología

Se siguieron las siete fases de la clase 15 en un tablero de GitHub Projects con estados Backlog, To Do, In Progress, In Review y Done. Cada requisito se cerró con una prueba automática que lo comprueba, y cada cifra de los reportes se calculó primero a mano.

Tabla 3

Fases del trabajo

Semana	Fase	Resultado
15	1 Organización · 2 Requerimientos y modelo · 3 Sistema base	BRIEF, REQUERIMIENTOS, MODELO_DATOS; actividades 1–4 integradas
16	4 Módulo 1 · 5 Módulo 2 + API + interfaz asíncrona	Liquidaciones y pedidos con pruebas propias
17	6 Seguridad 100 % y publicación	Auditoría 18/18; paquete para InfinityFree probado en una copia
17–18	7 Manual, informe, bitácora y defensa	Este informe, el manual y la presentación

Requerimientos

Tabla 4

Requerimientos de los módulos nuevos y no funcionales

Id	Requerimiento	Módulo
RF09	Registrar acopios con control de calidad (entregadas, aprobadas, motivo)	Módulo 1
RF10	Calcular liquidado, pagado y saldo de cada socia	Módulo 1
RF11	Registrar pagos sin superar el saldo y con referencia única	Módulo 1
RF12	La socia consulta solo su liquidación (web y API)	Módulo 1
RF13	Registrar pedidos que reservan stock (web, JSON/XML o API)	Módulo 2
RF14	Confirmar, entregar (venta en una transacción) o cancelar (libera reserva)	Módulo 2
RF15	Impedir vender o editar el stock por debajo de lo reservado	Módulo 2
RNF01	Seguridad: preparadas, hash, CSRF, cookie segura, CSP, roles, archivos internos bloqueados	Todo
RNF02	Integridad: 18 claves foráneas, UNIQUE, CHECK y transacciones con FOR UPDATE	Todo
RNF03	Privacidad: los datos públicos no muestran CI ni teléfono	Todo

Nota. La lista completa (RF01–RF15, RNF01–RNF06) está en docs/proyecto-final/REQUERIMIENTOS.md.

Diseño

Arquitectura

Aplicación PHP sin framework, organizada en capas: páginas en `public/` (lo único visible por URL), lógica en `src/` (seguridad, ventas, pedidos, liquidaciones, intercambio, API y repositorios con PDO) y plantillas en `plantillas/`. La vitrina se dibuja en el servidor y se mejora con fetch. En Apache, un `.htaccess` en la raíz deja ver solo `public/` y otro enruta la API.

Modelo de datos

Figura 1

Modelo entidad-relación (13 tablas, 18 claves foráneas)

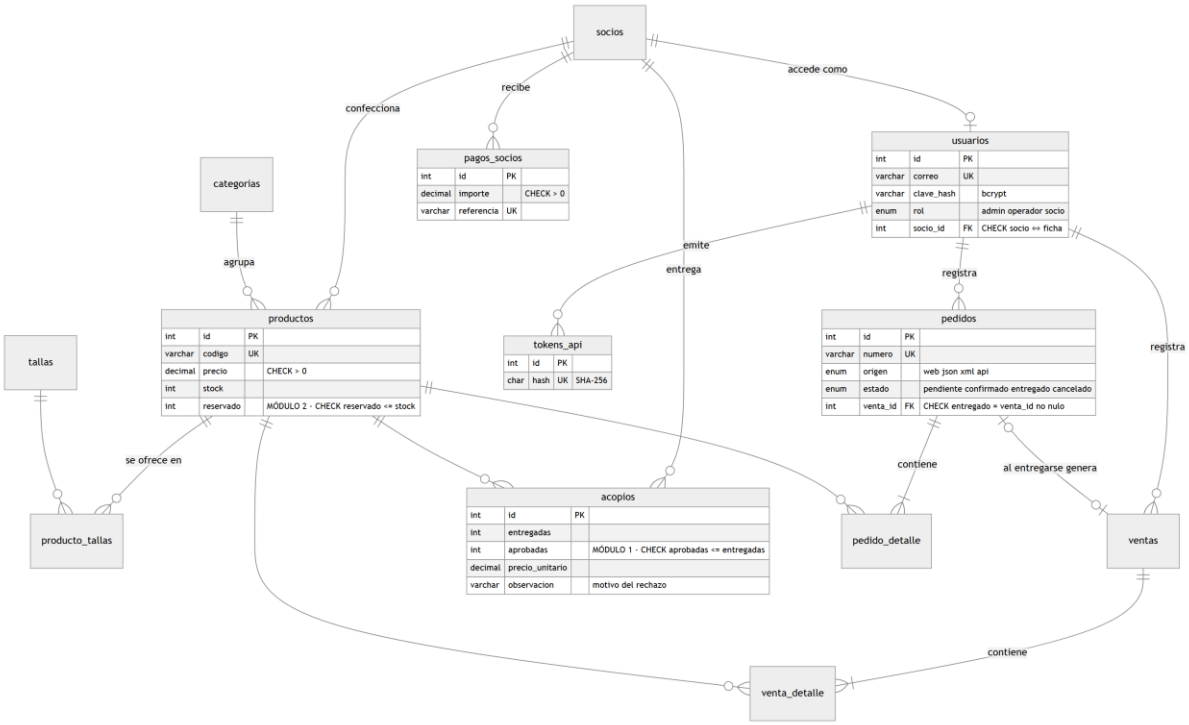


Tabla 5

Decisiones de diseño de la base de datos

Decisión	Motivo

El saldo se calcula, no se guarda	Nunca queda desactualizado
Subconsultas separadas para acopios y pagos	Un JOIN de ambos duplicaría los montos
Columna reservado con CHECK (reservado <= stock)	La base rechaza una reserva imposible aunque falle la aplicación
Entregar = venta + stock + reserva en una transacción con FOR UPDATE	Dos operadores no venden la misma unidad
Precio guardado en cada detalle	Lo vendido no cambia si cambia el catálogo
ON DELETE RESTRICT en ventas, acopios y pedidos	No se pierde el historial
CHECK usuario socio ⇔ ficha de socia	Una socia siempre ve solo sus propios datos

Implementación de los módulos

Módulo 1: liquidación y pagos con control de calidad

`registrar_acopio()` exige el motivo cuando se rechazan prendas, suma al stock solo las aprobadas y guarda el precio por prenda. `liquidaciones()` calcula entregadas, aprobadas, liquidado y pagado con subconsultas, y el saldo en centavos para evitar errores de redondeo. `registrar_pago_socio()` bloquea la fila de la socia, comprueba que el pago no supere el saldo y rechaza una referencia repetida. Una socia ve siempre su propia liquidación aunque cambie el parámetro de la URL, porque el filtro está en la consulta.

Figura 2

Liquidaciones vistas por la directiva, con el detalle de Rosa

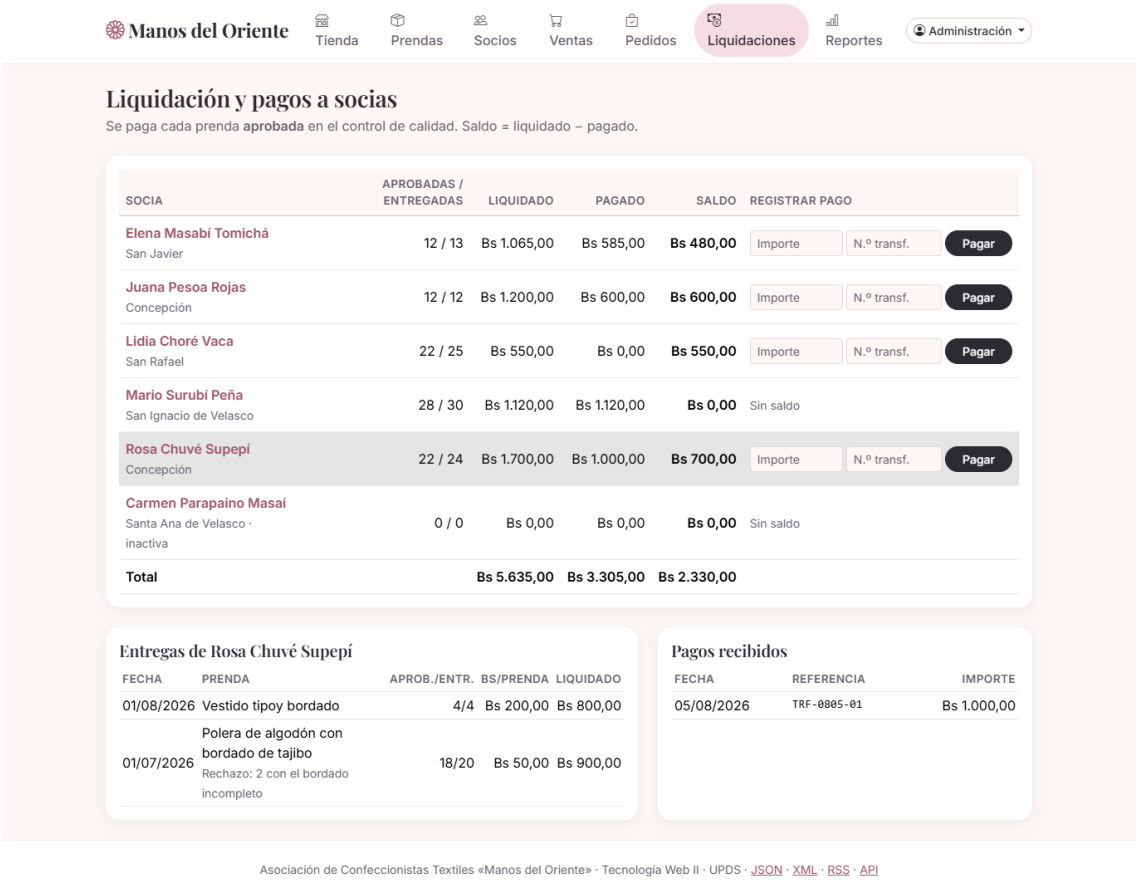


Figura 3

La misma información vista por la socia: solo la suya

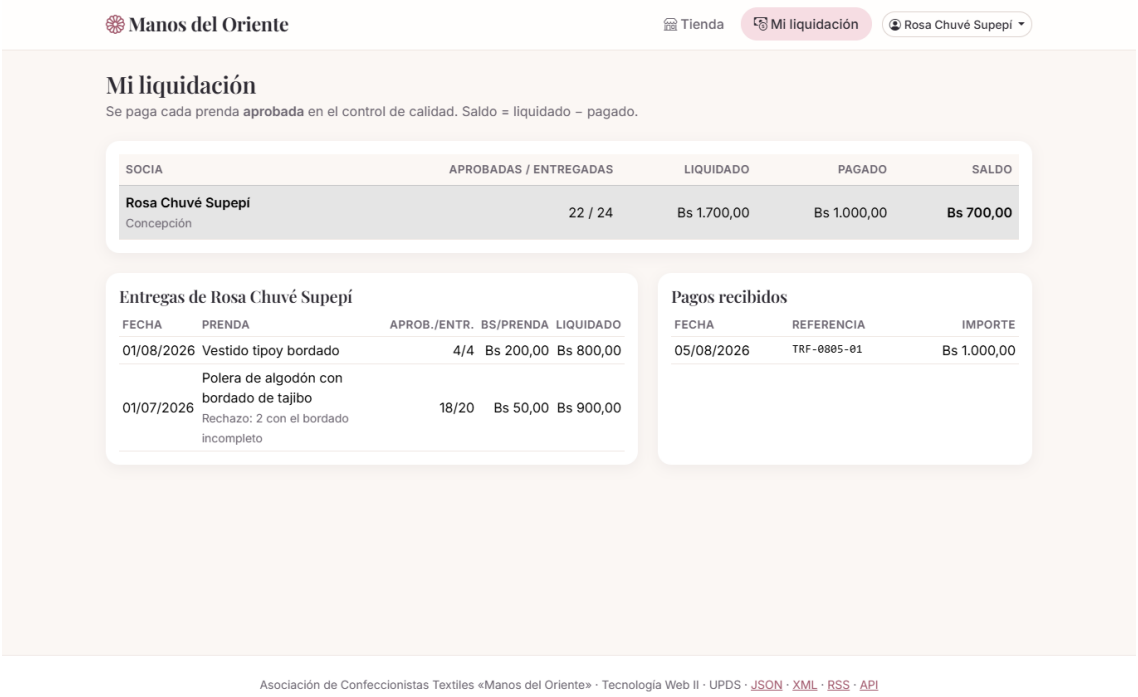


Tabla 6

Liquidaciones calculadas a mano con los datos de demostración (Bs)

Socia	Aprobadas / entregadas	Liquidado	Pagado	Saldo
Rosa Chuvé Supepí	22 / 24	1.700,00	1.000,00	700,00
Elena Masabí Tomichá	12 / 13	1.065,00	585,00	480,00
Mario Surubí Peña	28 / 30	1.120,00	1.120,00	0,00
Juana Pesoa Rojas	12 / 12	1.200,00	600,00	600,00
Lidia Choré Vaca	22 / 25	550,00	0,00	550,00
Total		**5.635,00**	**3.305,00**	**2.330,00**

Nota. Coinciden con la pantalla (Figura 2) y con las pruebas automáticas.

Módulo 2: pedidos de compradores con reserva de stock

`crear_pedido()` bloquea las prendas en orden de código (evita interbloqueos), comprueba que haya unidades libres (stock menos reservado), guarda el pedido y suma la reserva. `cambiar_estado_pedido()` respeta el ciclo de vida: al entregar llama a `insertar_venta()` con el precio pactado y baja stock y reserva a la vez; al cancelar libera la

reserva. Un pedido entregado o cancelado ya no cambia. Los pedidos pueden llegar por el formulario, por un archivo JSON o XML validado (laboratorio 12) o por `POST /api/v1/pedidos`.

Figura 4

Pedidos con su estado y el stock reservado

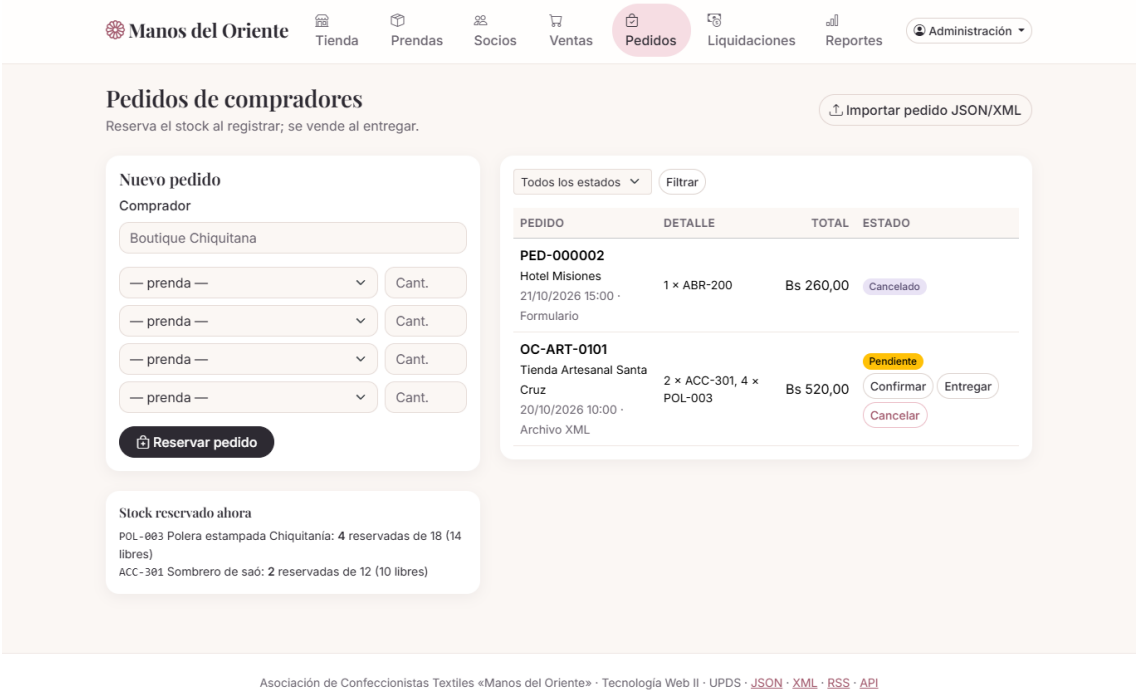


Figura 5

Importación de un pedido JSON o XML



Seguridad

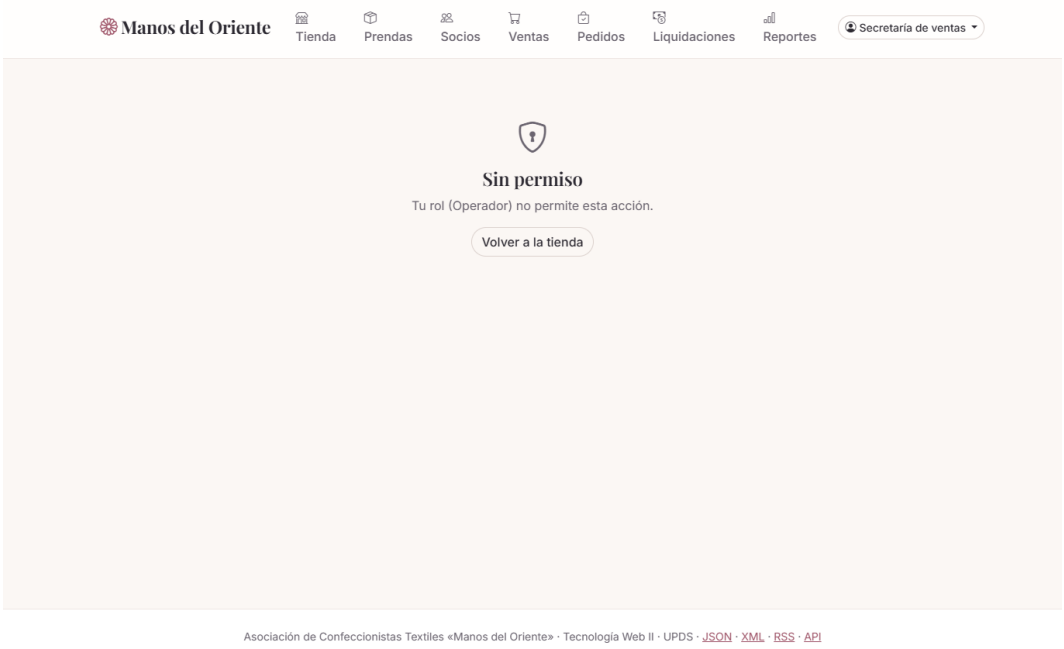
Tabla 7

Amenazas y defensas

Amenaza	Defensa	Dónde
Inyección SQL	Consultas preparadas reales; ORDER BY por lista blanca	src/Repositorio, src/*.php
Robo o fijación de sesión	Cookie HttpOnly, SameSite=Lax, Secure en HTTPS; id regenerado; 30 min	src/Seguridad.php
CSRF	Token en todos los formularios POST	csrf_campo(), exigir_csrf()
Fuerza bruta	Bloqueo de 15 min tras 5 intentos; mismo mensaje exista o no el correo	autenticar_usuario()
Acceso indebido	Permiso exigido en cada página y en cada operación de la API	PERMISOS, exigir_usuario()
XSS	e() en las vistas; textContent en JavaScript; CSP script-src 'self'	plantillas, public/js, arranque.php
Archivos internos	.htaccess: solo public/ es visible	herramientas/infinityfree/.htaccess
Robo de tokens	Solo el hash SHA-256 en la base; revocables	tokens_api, tokens.php
Credenciales en el código	config.php fuera de git; contraseñas nuevas al publicar	.gitignore, empaquetar_infinityfree.php

Figura 6

Un operador que intenta entrar a Usuarios recibe 403



Publicación

El script `herramientas/empaquetar_infinityfree.php` genera la carpeta htdocs con el código, los `.htaccess` y la configuración del hosting; un archivo SQL único para importar en phpMyAdmin (InfinityFree no permite crear la base desde PHP) y contraseñas aleatorias para los cuatro usuarios. Antes de subirlo se probó en una copia local que imita al hosting: Apache con `.htaccess` y la base importada desde ese mismo SQL. Allí funcionaron la tienda, la API con token Bearer y el ingreso con las contraseñas nuevas, y las de demostración fueron rechazadas.

InfinityFree presenta un desafío JavaScript a cada visitante nuevo. Un navegador lo resuelve solo; un cliente sin navegador necesita la cookie `__test`, por lo que el script de auditoría acepta esa cookie como parámetro. Los pasos detallados están en docs/proyecto-final/DESPLIEGUE_INFINITYFREE.md.

Pruebas

Tabla 8

Resultados de las pruebas

Prueba	Qué comprueba	Resultado
php verificar.php (MariaDB 10.11)	Actividades 2–4, módulos nuevos y seguridad del código	136 de 136
php verificar.php (MySQL 8.4)	Lo mismo en el otro motor	136 de 136
Navegador (Edge automatizado)	AJAX, autocompletar, cancelaciones, validación en vivo, roles, consola sin errores	12 de 12
herramientas/auditar.php (local)	Cabeceras, cookie, CSRF, rutas protegidas, archivos internos, API	18 de 18
herramientas/auditar.php (copia del hosting)	Lo mismo con Apache y .htaccess	18 de 18
PHP 8.2 (php -l)	Compatibilidad con la versión mínima del hosting	Sin errores

Figura 7

Salida de php verificar.php



Resultados y discusión

Tabla 9

Objetivos frente a resultados

Objetivo	Cómo se cumple	Evidencia
1 · 0 reclamos de pago	Cada rechazo queda con su motivo y la socia ve el cálculo	Figuras 2 y 3, Tabla 6
2 · 0 pedidos incompletos	Lo reservado no se puede vender ni bajar del stock	Módulo 2 en verificar.php
3 · Informe en 10 minutos	Reporte por fechas con mes, top 5 y por socia; CSV para Excel	docs/actividad-03
4 · Cada socia ve su saldo	Usuario propio con rol socia; web y API filtran por su ficha	Figura 3

Los objetivos se cumplen en el sistema; su efecto real sobre los reclamos y los pedidos solo se podrá medir cuando una asociación lo use. Las pruebas encontraron errores reales durante la integración: dos sesiones distintas que hacían perder al usuario, un error de variable que impedía entregar pedidos, un usuario vinculado a la socia equivocada, fotos asignadas a otras prendas y, en el paquete de publicación, un hash dañado al reemplazarlo y una regla que bloqueaba los estilos. Ninguno habría aparecido sin probar el sistema como lo usaría la asociación.

La principal limitación es el hosting gratuito: el desafío antibot complica las pruebas automáticas de la URL publicada y la extensión SOAP puede no estar disponible, aunque REST cubre la misma consulta.

Conclusiones

- Los dos módulos responden a discusiones concretas de la asociación, no solo agregan pantallas.
- Calcular el saldo en lugar de guardarlo y usar CHECK y transacciones deja la consistencia en manos de la base.
- La privacidad por diseño (filtro en la consulta y CHECK de la ficha) asegura que cada socia vea solo lo suyo.
- El sistema está listo para publicarse: el paquete se probó en una copia del hosting con auditoría al 100 %.
- Queda pendiente subirlo a InfinityFree, auditar la URL publicada y validarlo con una asociación real.

Referencias

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

MDN Web Docs. (s. f.). *Content Security Policy (CSP)*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP

Naciones Unidas. (s. f.). *Objetivo 8: Promover el crecimiento económico inclusivo y sostenible, el empleo y el trabajo decente para todos*. https://www.un.org/sustainabledevelopment/es/economic-growth/

Nottingham, M., Wilde, E., & Dalal, S. (2023). *Problem details for HTTP APIs* (RFC 9457). Internet Engineering Task Force. https://doi.org/10.17487/RFC9457

OpenAPI Initiative. (2021). *OpenAPI Specification v3.1.0*. https://spec.openapis.org/oas/v3.1.0

Oracle. (s. f.). *MySQL 8.4 reference manual: Locking reads*. https://dev.mysql.com/doc/refman/8.4/en/innodb-locking-reads.html

OWASP Foundation. (s. f.-a). *Cross-site request forgery prevention cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

OWASP Foundation. (s. f.-b). *Password storage cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Requena Llorentty, J. N. (2026a). *Guía integral del estudiante: Actividad 4, Tecnología Web II* [Material de clase]. Universidad Privada Domingo Savio.

Requena Llorentty, J. N. (2026b). *Clase 15: Arranque del Trabajo Final Integrador* [Diapositivas]. Universidad Privada Domingo Savio.

Anexos

Anexo A. Registro de uso de inteligencia artificial

El equipo usó Claude Code (modelo Claude Opus 5.5) para revisar el estado del proyecto, integrar las actividades 3 y 4, construir los módulos, las pruebas y los documentos. Cada propuesta se verificó con pruebas automáticas y cálculos a mano. El registro completo, con los errores de la IA que las pruebas detectaron y las decisiones del equipo, está en docs/BITACORA_IA.md; cada integrante completa allí su reflexión.

Anexo B. Repositorio y documentos

Tabla 10

Ubicación de los entregables

Recurso	Ubicación
Repositorio	https://github.com/TecnologiaWeb2/Tecnologiaweb2
BRIEF y KICKOFF	docs/proyecto-final/BRIEF.md, KICKOFF.md
Requerimientos y modelo	docs/proyecto-final/REQUERIMIENTOS.md, MODELO_DATOS.md
Plan y checklist	docs/proyecto-final/PLAN.md, CHECKLIST-TFI.md
Publicación	docs/proyecto-final/DESPLIEGUE_INFINITYFREE.md
Manual de usuario	docs/proyecto-final/Manual_Usuario_Manos_del_Oriente.pdf
API	docs/actividad-04/endpoints.md, docs/openapi.yaml
Pruebas	verificar.php, pruebas/